MoveIn Proof

Protect your rental bond

40005 Advanced iOS Applications
Assessment 4: App Store–Ready Production Solution

Oscar Costa
Student ID: 25859366
Individual submission

Repository: github.com/oscarcosta-git/MoveInProof
Walkthrough video: youtu.be/KMVMSueb6ac

Section 1 - Problem, Stakeholders and Competitors

Oscar Costa - 25859366

Problem

When you move into a rental in NSW you have to fill in a condition report and send it back to the agent, usually within a week. At the end of the lease, if the landlord claims part of your bond for damage, the only way to argue it is to show the damage was already there when you moved in. If it goes to NCAT, the tenant has to back up their side with evidence (NSW Government, n.d.).

In practice most renters just take a few photos on their phone and fill in the form by hand. A year later those photos are buried in the camera roll, not sorted by room, and the dates can be lost if they were sent over messaging apps. So when the bond is claimed, the tenant often has nothing solid to show.

Who it's for

The target user is a first-time or student renter in Sydney, renting through an agent, often with housemates. Their bond is four weeks' rent, which is a lot of money for them. They're comfortable with an iPhone but don't know what kind of evidence would actually hold up in a dispute.

Evidence

- 31% of Australian households rented at the 2021 Census, about 2.9 million households (AIHW, 2025). In NSW the number of renting households grew by over 17.5% between 2016 and 2021 (Mowbray, 2022).
- A Finder survey of 2,003 Australians found over 30% of people who had rented and moved in the last five years lost some or all of their bond, with damage the most common reason (Whitten, 2018).

Competitors

	Resident Inventory	**zTenant**	**MoveIn Proof**
Who can use it	Tenants whose agency uses it	Tenants invited by their property manager	Any tenant
Account	Login required	Invite and login required	None
Data	Report sent to the property manager; collects user and device IDs	Collects location, name and email	Stays on the phone, nothing collected
Timestamp on photo	Not mentioned	Not mentioned	Burned into every photo

Both apps are free and do the basics well (photos by room, a report at the end), but they're built for property managers. The tenant can't just download one and use it, and the report goes to the agency. The other option, the Camera app plus the paper form, has no structure at all.

MoveIn Proof is aimed at the tenant. There's no account, nothing leaves the phone, every photo has the date and time stamped on it, and the tenant gets their own PDF report. The widget also keeps the report due date on the Home Screen.

Why iOS

The main job is taking photos in each room, so it needs to be a phone app. It works offline (new rentals often don't have internet yet) because everything is stored on the device with SwiftData, and the photos are protected by iOS file encryption. WidgetKit shows the key dates on the Home Screen, and the share sheet makes it easy to send the PDF to the agent.

References

Australian Institute of Health and Welfare. (2025). *Home ownership and housing tenure*. https://www.aihw.gov.au/reports/australias-welfare/home-ownership-and-housing-tenure

Cloud Advantage Services. (2025). *Resident Inventory* [Mobile app]. App Store. https://apps.apple.com/us/app/id1422378616

Mowbray, J. (2022, July 4). *Census 2021: Renters are the fastest growing tenure in Australia*. Tenants' Union of NSW. https://www.tenants.org.au/blog/census-2021-renters-are-fastest-growing-tenure-australia

NSW Government. (n.d.). *Dealing with bond disputes for tenants*. Retrieved October 30, 2026, from https://www.nsw.gov.au/housing-and-construction/renting-a-place-to-live/residential-rental-bonds/dealing-bond-disputes-for-tenants

Scaled Tech Inc. (n.d.). *zTenant* [Mobile app]. App Store. https://apps.apple.com/kr/app/ztenant/id1491039980?l=en-GB

Whitten, R. (2018, June 14). *Renters, 30% of you won't get your bonds back*. Finder. https://www.finder.com.au/30-of-aussie-tenants-dont-get-their-rental-bond-back

Section 2 - App Store Metadata & Product Strategy

The App Store Connect metadata package and product strategy. Character counts are checked against Apple's limits.

App identity

Field	Value	Count / limit
App Name	MoveIn Proof: Condition Report	30 / 30
Subtitle	Protect your rental bond	24 / 30
Bundle ID	com.oscarcosta.MoveInProof	
Version	1.0	
Price	Free	
Primary category	Productivity	
Secondary category	Lifestyle	

Why these categories: the app's core job is producing a document (a condition report) quickly and accurately, which puts it in **Productivity**. Renters browsing for home and moving tools look in **Lifestyle**, so the secondary category puts it where they look.

Naming: "MoveIn Proof" is short and distinctive, and says what the app gives you: proof of the property's condition when you moved in. "Condition Report" after the colon adds the term renters actually search for. The name uses no trademarks, competitor names or generic-only words, in line with App Store Review Guideline 2.3.7.

Age rating

Rating: 4+

Questionnaire item	Answer	Justification
Violence, sexual content, profanity, horror, drugs, alcohol, gambling	None	The app contains no such content.
Medical or treatment information	None	No health data is collected or shown.
Unrestricted web access	No	The app has no web views and makes no network requests.

User-generated content shared with others	No	Photos and notes stay on the device. Nothing is published, uploaded or visible to other users, so no moderation is needed (Guideline 1.2). The user can choose to share their own PDF through the system share sheet.
Location sharing	No	The app doesn't request or use location.
Contests, chat or messaging	No	

Content rights: the app contains no third-party content. All text, icons and illustrations are original or Apple SF Symbols, and there are no third-party packages.

Keywords

tenant,lease,inspection,move in,move out,renter,landlord,deposit,damage,photo,checklist,apartment

97 / 100 characters. Words already in the name and subtitle (condition, report, rental, bond, proof, protect) are left out, because Apple indexes those separately and repeating them wastes space.

Promotional text

Moving in? Photograph every room with date and time stamps, flag existing damage, and export a professional condition report to protect your bond. No account needed.

165 / 170 characters. Promotional text can be updated without a new app review, so it can mention seasonal moving periods later.

Description

2,046 / 4,000 characters.

Your condition report is your best evidence when your bond is on the line. MoveIn Proof helps you document a rental property properly on the day you move in, so you can prove what was already there when you move out.

WALK EVERY ROOM, IN ORDER
• A guided checklist of rooms, from the entry to the balcony. Add, remove or reorder rooms to match your place.
• See at a glance which rooms are done, which is up next and how many photos you've taken.

TIMESTAMPED PHOTO EVIDENCE
• Take photos in the app with your iPhone's camera. Each one has the date and time stamped into the image itself, so the timestamp stays visible wherever the photo is shared.
• Choose which wall or area you're photographing, and use flash or tap to focus.

FLAG DAMAGE AS YOU GO
• Tap Scuff, Crack, Stain or More before a shot to flag that photo as existing damage.

• Add a damage type, severity (minor, moderate or major) and notes for every issue.
• Flag any earlier photo as an issue later.

NEVER MISS A KEY DATE
• Track when your condition report is due, when your move-out inspection is booked and when your lease ends.
• Add the Tenancy dates widget to your Home Screen to see the days remaining.

A PROFESSIONAL REPORT IN ONE TAP
• Export a clean A4 PDF with a cover summary, every room's issues and a grid of timestamped photos.
• Share it by email, AirDrop or Messages, print it, or save it to Files.
• Include or leave out photos, or export only the rooms with flagged issues.

PRIVATE BY DESIGN
• No account, no sign-up and no cloud. Your tenancy, photos and notes are stored only on your iPhone.
• No ads, no tracking and no analytics.
• Works fully offline.

BUILT FOR IPHONE
• Light and Dark Mode.
• Supports Dynamic Type, including the largest accessibility text sizes.
• Clear guidance if camera access is turned off, with a shortcut to Settings.

MoveIn Proof doesn't replace your state's official condition report form or legal advice. It gives you organised, timestamped evidence to attach to your report and to refer back to at the end of your lease.

What's New (version 1.0)

First release. Photograph each room with timestamped photos, flag existing damage, track your key lease dates and export a PDF condition report.

URLs

Field	URL
Support URL	https://oscarcosta-git.github.io/MoveInProof/support
Marketing URL	https://oscarcosta-git.github.io/MoveInProof
Privacy Policy URL	https://oscarcosta-git.github.io/MoveInProof/privacy

These are planned GitHub Pages addresses for the project. They aren't live yet; a real submission would publish these pages first, because App Review checks that the support and privacy URLs

load.

Privacy

Privacy policy (summary for the policy page)

MoveIn Proof doesn't collect any personal data. Everything you enter (tenancy details, room photos, issues and notes) is stored only on your iPhone, in the app's private storage, with iOS file protection. Nothing is sent to us or to any third party. The app has no accounts, no analytics, no advertising and makes no network requests. Deleting the app, or using "Delete all data" in the app, removes everything. If you choose to share a PDF report, it goes only where you send it using the iOS share sheet.

App Privacy "nutrition label"

Section	**Declaration**
Data Used to Track You	None
Data Linked to You	None
Data Not Linked to You	None
Overall label	**Data Not Collected**

Storage: on-device only (SwiftData store and photo files in Application Support), not synced. The Home Screen widget reads a small summary (address, three dates and the issue count) from an App Group shared only between the app and its own widget on the same device.

Permission usage strings

Key	**Used**	**String shown to the user**
NSCameraUsageDescription	Yes	"MoveIn Proof uses the camera to photograph the condition of each room."
NSPhotoLibraryUsageDescription	No	Photos are taken in the app and stored privately, never read from the photo library.
NSLocationWhenInUseUsageDescription	No	Location isn't used. The address is typed by the user.
NSFaceIDUsageDescription	No	Not used.

If camera access is denied, the app shows "Camera access is off" with an **Open Settings** button, and it never crashes or loops on the permission prompt.

Privacy manifest

PrivacyInfo.xcprivacy is included in both the app and the widget extension:

- NSPrivacyTracking: false, with no tracking domains

- NSPrivacyCollectedDataTypes: none
- Required reason API: UserDefaults, reason 1C8F.1 (reading and writing data in an App Group shared only with the app's own widget)

App Review notes

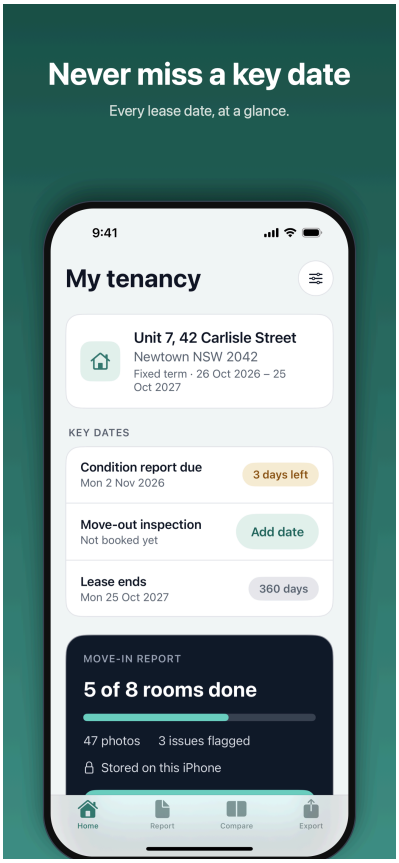
No account or login is required. All features work offline in the Simulator or on a device.
To try the app, complete the short setup screen (enter any street address and tap Start move-in report), then use Report > Start a room to take photos.
The Simulator has no camera, so the capture screen shows a drawn wall that is saved and timestamped like a real photo. On a device, the back camera is used.

Product strategy

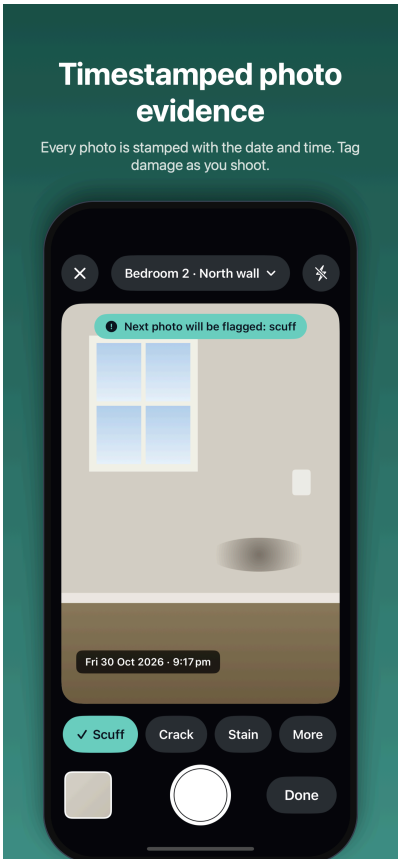
- **Business model:** free in version 1.0, to build a user base and reviews among renters during the high-turnover moving season. A possible future one-off "Pro" in-app purchase could add move-out comparison and multiple tenancies. The core move-in report stays free, which is also a reason renters would pick it over subscription competitors.
- **Target audience:** Australian renters, especially first-time and student renters, moving into a new rental property who need to complete and return a condition report within the deadline.
- **Positioning:** a renter-first, private, offline tool. Unlike property-manager inspection apps, it's built for the tenant's side, needs no account and keeps the evidence on the tenant's own phone.
- **Screenshots:** six framed 1320×2868 images, shown on the next page.

App Store screenshots

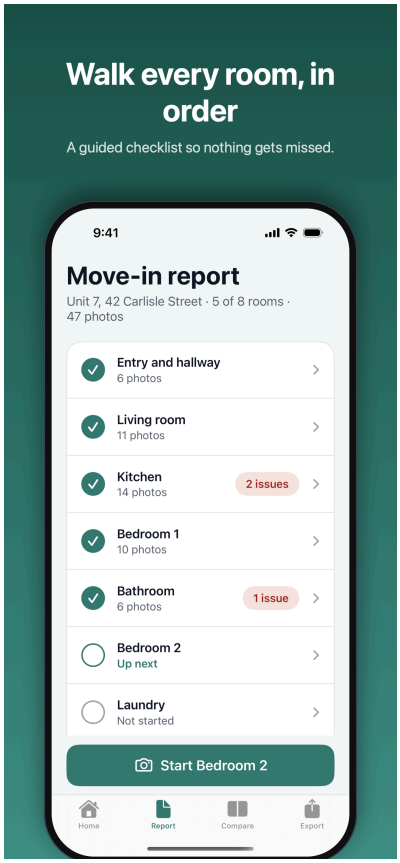
Six screenshots at 1320 × 2868 px (6.9-inch iPhone), with no transparency. The last one shows Dark Mode.



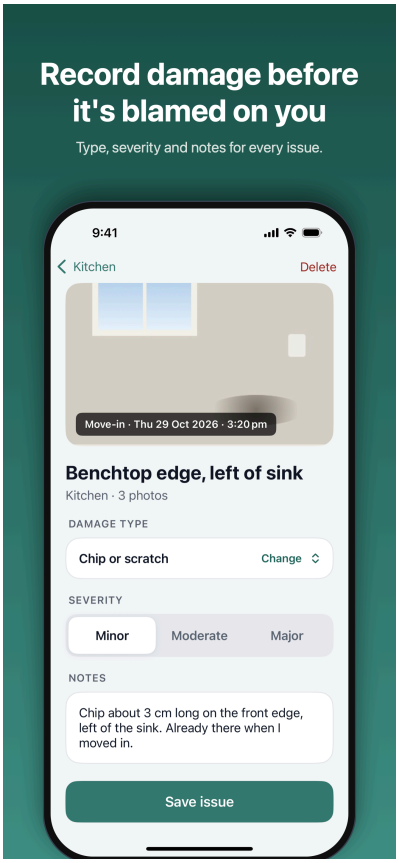
Key dates



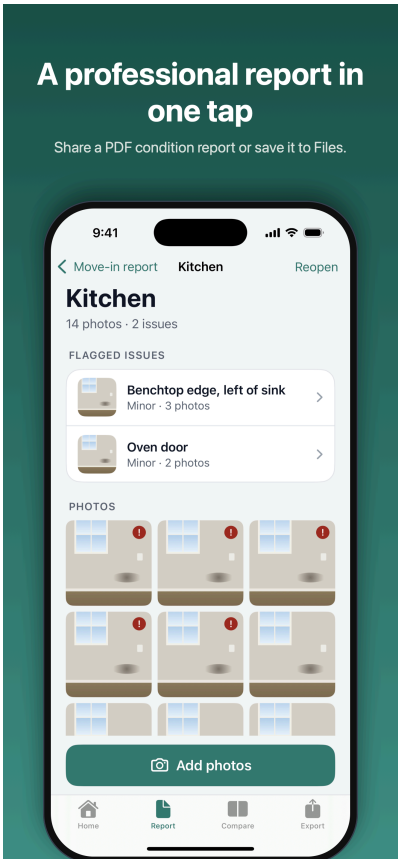
Timestamped photos



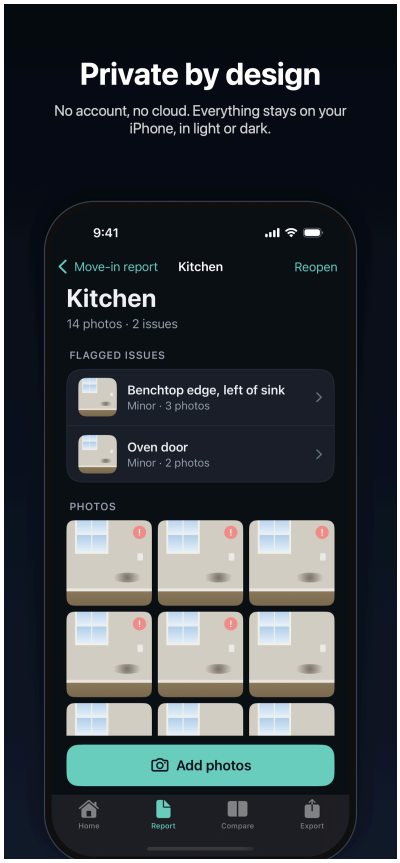
Room checklist



Flag damage



PDF report

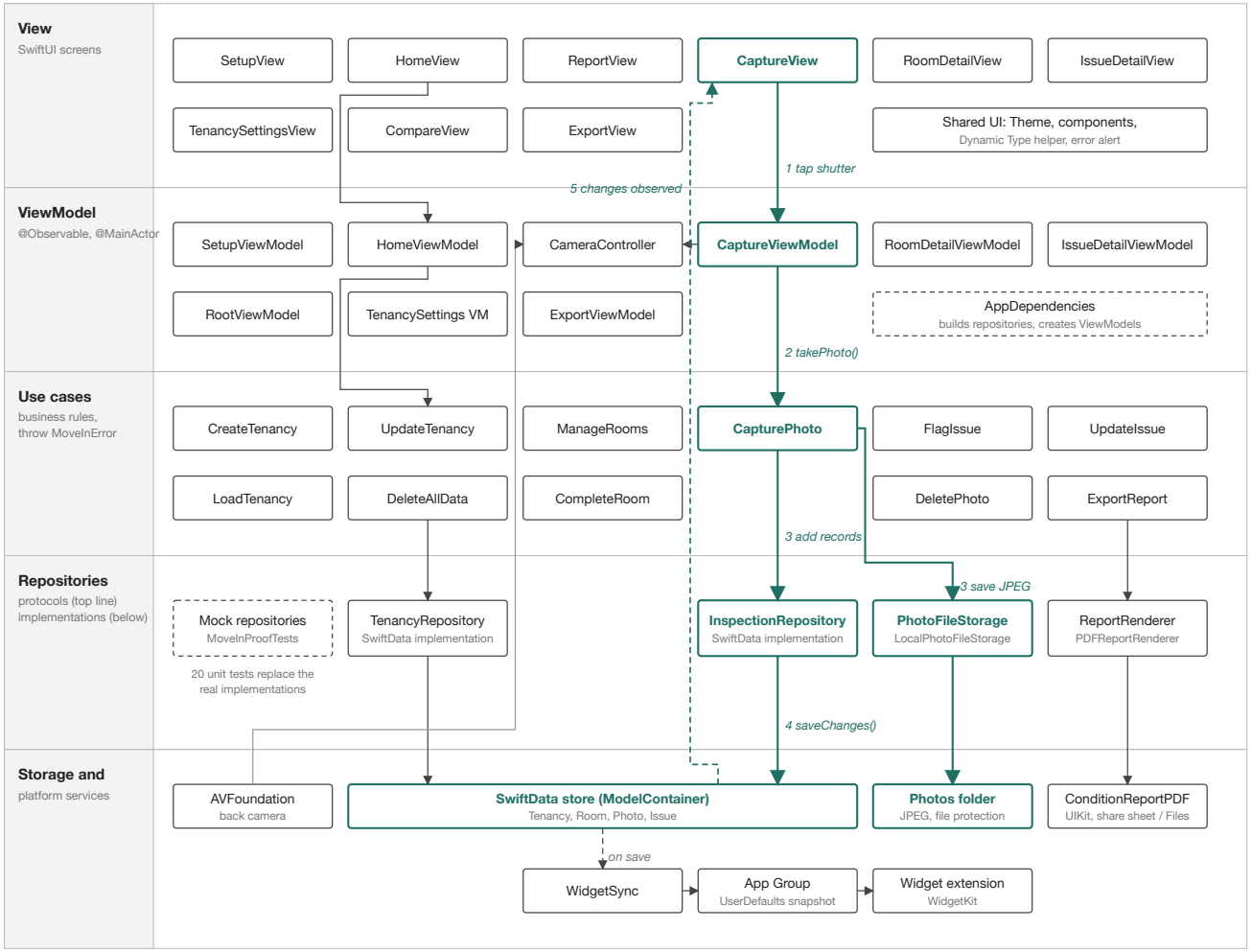


Dark mode

Section 3 - System Architecture & Data Flow Diagram

MoveIn Proof: architecture and data flow

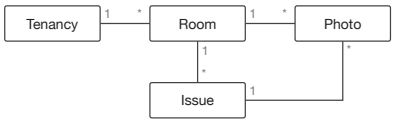
MVVM + use cases + repositories. SwiftUI, SwiftData, AVFoundation, WidgetKit. All storage is on the device.



Taking a photo

1. CaptureView sends the tap to CaptureViewModel.
2. The ViewModel captures the image with CameraController, stamps the date and time on it and calls CapturePhoto.
3. CapturePhoto saves the JPEG through PhotoFileStorage and adds the Photo (and an Issue if tagged) through InspectionRepository.
4. saveChanges() commits to SwiftData. If that fails the JPEG is deleted and MoveInError.storageFailed is thrown.
5. SwiftData observation updates the screen. Errors are shown as an alert by the ViewModel.
   WidgetSync also writes a snapshot for the widget.

Data model



Deleting a room cascades to its photos and issues.
Deleting an issue keeps its photos.

Notes

Each layer only calls the one below it.
Views never import SwiftData; use cases only see repository protocols.
UI state runs on the main actor; the camera session has its own serial queue.
No network access. The widget reads a copy of the key dates from the App Group.

Key

- calls / depends on
- path traced above
- notification / observation
- wiring or test-only component

Section 4 - Team Contribution & Governance Matrix

Team

Name	Student ID	GitHub
Oscar Costa	25859366	oscarcosta-git

This project was completed individually, with no other team members. AI assistance is declared in Section 5.

Git commit verification

All commits are by Oscar Costa (student ID 25859366). Git author names **SlurpyDevv** and **Oscar Costa** are the same person, using the same email; merge commits from GitHub pull requests appear under the GitHub account **oscarcosta-git**.

Commit	Date	Message
8c65034	2026-10-22	MoveInProof Frontend UI
5669dfa	2026-10-22	Frontend UI: all screens from mockups with placeholder data
b331af1	2026-10-22	Widgets UI: Tenancy dates widget in small and medium sizes
2bd74a3	2026-10-29	Local storage: save tenancy, rooms, photos and issues on device
5c2342d	2026-10-29	Camera: live capture with timestamped photos, flash and tap to focus
b076242	2026-10-30	PDF export: condition report with timestamped photos, share and save to Files
a6bee99	2026-10-30	Widget data: show the saved tenancy dates via a shared App Group
d2ab9e1	2026-10-30	refactor: adopt MVVM with use case and repository layers
166ce23	2026-10-30	test: add unit test target with mock repositories for domain use cases
ee35bf8	2026-10-30	feat: support Dynamic Type across the app
51487eb	2026-10-30	feat: add app icon and privacy manifests
9891daf	2026-10-30	docs: add README
778c9f8	2026-10-30	fix: make damage tagging manual and remove simulated detection
c7b6830	2026-10-30	chore: date sample data relative to today
98b74d4	2026-10-30	docs: add App Store screenshots
842e497	2026-10-30	docs: add App Store metadata draft
1f15fe1	2026-10-30	docs: add architecture and data flow diagram

Governance

- **Branching:** main (release), develop (integration) and feature branches (feature/..., feat/..., fix/..., docs/...). Work reached main only through GitHub pull requests.
- **Commit style:** Conventional Commits (feat:, fix:, refactor:, test:, docs:, chore:) from the architecture work onward; earlier feature commits predate this.
- **Quality gate before merging:** zero build warnings, all unit tests passing, and a manual check in the iOS Simulator.
- **AI assistance:** declared in Section 5 and the README.

Section 5 - Reflective Report

Technical reflection

I did this project on my own, so this replaces the group reflection.

The biggest trade-off was the architecture. My first working version had the SwiftUI views saving straight to SwiftData. It worked, but the rules were mixed into the views and I couldn't test any of it, and it broke the brief's rule about no database calls in views. Near the end I changed it to MVVM with use cases and repositories. Each screen got a ViewModel, the rules moved into use cases (a lease can't end before it starts, a room needs a photo before it's marked done, no duplicate room names), and the use cases only talk to repository protocols. It was a lot of rework late in the project, but afterwards I could write 20 unit tests using mock repositories, which I couldn't do before. I also added an AppDependencies class that creates the repositories once and gives each screen its ViewModel, so none of the views need to know SwiftData exists.

I kept the SwiftData model classes as my domain models instead of making separate structs for them. That saved a lot of code, but it means the domain still depends on SwiftData, and the tests need an in-memory container to create objects. For an app this size I think it was fine, but I'd change it if the app got bigger.

I also had to decide where the data lives. I kept everything on the phone with no account and no cloud sync. That means the app works offline (new rentals often don't have internet yet), photos of someone's home never leave their phone, and the App Store privacy label can say "Data Not Collected". The downside is there's no backup, so if the phone is lost the evidence is too.

Apple's guidelines changed a few things. The only permission I ask for is the camera. If the user says no, the camera screen tells them access is off and gives them a button to open Settings, instead of crashing or asking again. I added privacy manifests to the app and the widget. For errors, every use case throws one error type with a plain message, so the user sees something like "The photo couldn't be saved" in an alert instead of a SwiftData error.

Guideline 2.3, about accurate metadata, made me remove a feature. My mockups had the camera showing "Possible scuff" with a box around the damage, like it was detecting damage with AI. It wasn't. It was faked in the Simulator. I took it out and made tagging manual: you tap Scuff, Crack or Stain and a banner says the next photo will be flagged. It's less impressive, but it's honest, and the App Store description only says what the app actually does.

I also added Dynamic Type. At the biggest accessibility sizes some layouts broke, so I made some rows stack vertically, and the severity picker stacks too so "Moderate" doesn't split across two lines. I found these by checking the main screens at the largest text size in the Simulator.

The hardest problem was the widget. A widget runs separately from the app, so it can't read the app's database. I used an App Group: when the app saves, it writes a small summary (address, report due date, lease end and number of issues) that the widget reads, then tells the widget to refresh. That worked in the Simulator, but then I found out App Groups don't work with a free Apple developer account, so I can't install that version on my own phone. Since the brief doesn't need a

paid account, I demo the widget in the Simulator. The widget also had to make sense before any tenancy was set up, so it shows "Add your tenancy" until the app has saved one.

Testing found two smaller bugs. Photo thumbnails showed grey boxes because the "Application Support" folder has a space in its name and the file path was being encoded wrong. And in the PDF, photos were cropped to fill each square, which cut off the date and time stamped in the corner. The timestamp is the whole point of the evidence, so I changed it to show the full photo. Neither of these showed up as an error when building. I only found them by using the app and looking at what it actually saved.

I also had problems with Git. During the refactor GitHub Desktop switched branches while I had changes that weren't committed, and the work ended up on the wrong branch. I had to move it back and redo two files. After that I kept to feature branches, merged into develop and then main through pull requests, used Conventional Commit messages, and only merged when the build had no warnings and all the tests passed.

Individual reflection

The idea for MoveIn Proof and the screen designs were mine, and I decided what the app needed to do. I built it with Claude Code. I'd explain what a feature should do, then test it in the Simulator and ask for changes when something was wrong, like the timestamps getting cut off in the PDF, or the fake damage detection, which I decided to remove.

The main thing I learned was why the architecture matters. Before this, MVVM, use cases and repositories were just words to me. Seeing my app go from views saving straight to the database, to a version where the rules are in one place and can be tested, made it make sense. I also learned how SwiftData relationships work, how a widget gets data from its app, and that some Apple features depend on what kind of developer account you have.

My biggest personal struggle was Git. A few times I undid commits or switched branches at the wrong time in GitHub Desktop and lost track of where my work was. I got better by slowing down and keeping each feature on its own branch.

Testing taught me a lot too. Some bugs only showed up when I actually used the app, like grey boxes instead of photos, so I learned not to trust a build just because it succeeds.

Next time I'd set up the architecture and tests at the start instead of near the deadline, and test on my real phone earlier. I'd also commit more often so my Git history is cleaner.

Oscar Costa Student ID: 25859366 30 October 2026

AI assistance declaration

I used Claude Code (Anthropic) to help write the code and tests, fix bugs, and draft parts of this document, including a first draft of this reflection. I reviewed and tested everything and take responsibility for what's submitted.